Technical Report: Apertus Sentinel

Automated AI Security & OWASP Code Auditor

Overview

Apertus Sentinel is an automated static code analysis and auto-remediation framework built on Apertus-v1.5-8B, served through the CSCS inference API. It audits Python code for OWASP Top 10 flaws and also checks for prompt injection in agent code (e.g., Command Injection, SQL Injection, Hardcoded Secrets) and emerging AI threat vectors (e.g., Indirect Prompt Injection / Context Infiltration).

The system features an interactive command-line interface (CLI) that automatically extracts refactored, secure code blocks generated by the model and writes them to a dedicated output directory (`patched_files/`). Furthermore, it computes a score per run and writes a Markdown badge into the report so the report shows pass/fail at a glance.

Project Metric	Specification
Project Title	Apertus Sentinel — Automated AI Security & OWASP Code Auditor
Track	Track 2B – Own Project
Base Model	`swiss-ai/Apertus-v1.5-8B`
Inference Infrastructure	CSCS (Swiss National Supercomputing Centre) Inference API
Repository	GitHub - Deivanai0610/apertus-sentinel

System Architecture & Workflow

Apertus Sentinel operates through a modular 4-stage processing steps:

1. Batch Target Ingestion

- The engine recursively ingests source files from a target directory (`test_cases/`).
- Standardizes file formatting and contextual metadata like wrapping each file with its filename and a fixed instruction header before dispatching prompts.

2. Inference & Structured Analysis (CSCS API)

- Sends each file to `swiss-ai/Apertus-v1.5-8B` hosted on the CSCS Inference API endpoint.
- The prompt asks the model for four fields: vulnerability type, severity, root cause, and a patched version.

3. Dynamic Security Scoring & Badge Generation

- Evaluates the proportion of severe flaws found relative to total audited files.
- Dynamically constructs Markdown shield badges (e.g., `Apertus Security Score: 0%` for unpatched high-risk targets) embedded directly into the final audit summary report (`FINAL_SECURITY_AUDIT_REPORT.md`).

4. Interactive Auto-Fixer & Patch Export

- Prompts the user via CLI to automatically generate patched files.
- Upon confirmation, parses raw LLM patch responses, strips markdown delimiters, and writes secure scripts to `patched_files/`.

Benchmark Evaluation & Test Cases

Apertus Sentinel was evaluated across a 5-file benchmark suite five test files: three classic OWASP cases, one business-logic bug, one prompt injection case:

Target File	Primary Vector	What's wrong with the file?
`sql_injection.py`	OWASP A03: Injection	Dynamic string concatenation in SQL queries leading to database compromise.
`command_injection.py`	OWASP A03: Injection	Unsanitized OS execution via `os.system` / `subprocess`.
`hardcoded_secret.py`	OWASP A07: Auth Failures	Exposed API credentials and secret tokens stored in plain text.
`bank_payment_hijack.py`	Broken Business Logic	Unvalidated financial transaction logic lacking multi-factor authorization and account verification.

Target File	Primary Vector	What's wrong with the file?
`context_infiltration.py`	OWASP LLM01: Prompt Inje...	Indirect Prompt Injection targeting autonomous agent workflows through untrusted external context.

How to Run

1. Prerequisites

Install the required standard dependency:

```
pip install openai
```

2. Configure API Endpoint

In `auditor.py`, configure the CSCS Inference API client credentials and base URL.

3. Execute Audit & Auto-Patching

Run the audit engine via the command line:

```
python auditor.py
```

When prompted, enter `y` to execute automated code patching and export secure scripts to patched_files/.